

<pre> class WorldLoader: def __init__(self): self.classname = "WorldLoader" self.description = "This module loads a world environment from a specified .blend file." self.input = "blend_file_path: The file path of the .blend file\n \ "world_name: The name of the world environment to load" self.obj_name = None self.limitation = None @staticmethod def run(strength: Annotated[str, f"Light intensity, ranging from 0.1 to 0.5"], sun_elevation: Annotated[str, f"Sun altitude angle, ranging from 1 to 90"]) -> Annotated[str, "result."]: # try: # Load "World" data block from the specified .blend file path = os.path.join(os.getcwd(), "C:blender_file/world_2.blend") with bpy.data.libraries.load(path) as (data_from, data_to): data_to.worlds = data_from.worlds </pre>	
<pre> class snap_obj_to_ground: def __init__(self): self.classname = "snap_obj_to_ground" self.description = "Map the placed object vertically to the supporting object." self.input = ""placed object name and supporting object name"" self.obj_name = None self.limititon = "The placed object must be above the supporting object" @staticmethod def run(surface_obj_name: Annotated[str, f"The name of the thing that holds the placed object"], placed_object_name: Annotated[def choose_object(name): target_entity = bpy.data.objects.get(name) if target_entity: bpy.context.view_layer.objects.active = target_entity target_entity.select_set(True) bpy.ops.object.select_all(action='DESELECT') return target_entity def get_raycast_param(view_layer): if bpy.app.version >= (2, 91, 0): return view_layer.depsgraph else: return view_layer </pre>	
<pre> class BuildingAdd: def __init__(self): self.classname = "BuildingAdd" self.description = "Add and place building assets based on information" self.input = ""The json file path that records the information."" self.obj_name = None self.limititon = None @staticmethod def run(json_path: Annotated[str, f"The json file path that records the information"]): with open(json_path, 'r', encoding='utf-8') as f: data = json.load(f) building_obj_names = data.get('building_obj_name_list', []) collections = ["AmericaVillage"] for obj_name in building_obj_names: asset_library_path, collection_name, center, angle = buildmatch(obj_name, collections) if not asset_library_path: continue load_collection(asset_library_path, collection_name, obj_name+'.'+collection_name) loaded_collection = bpy.data.collections.get(obj_name+'.'+collection_name) </pre>	

Figure 11: Standardized python template for PCG module documentation.

Template:

Role: You serve as a Blender API operator to choose the feasible parameter value according to the description.

Task: Your task is to break down the user's input into the parameters required for generating terrain mesh. The API that you can invoke for this purpose is called "Terrain_mesh_generate".

Document:

[{Name: Rotate_Slope, Description: This parameter allows you to define the magnitude of the terrain slope, Value: -6.280 – 6.280, Default: 0.000},

{Name: Flat_Floor, Description: This parameter allows you to flatten the ground, Value: [True, False], Default: False},

{Name: Sink, Description: You can change the elevation or height of the terrain, Value: 0.000 – 1.000, Default: 0.500},

{Name: Add_Valleys, Description: Based on the input provided by the user, speculate on whether the addition of valleys is necessary., Value: [True, False], Default: False},

{Name: Scale_Valleys, Description: Based on the user input, speculate on the scaling factor for the valleys., Value: 0.000 – 5.000, Default: 1.000}, ...]

Format: Out put the modified api in the follow form:

[Rotate_Slope: You need to give a parameter value in the range [-6.280 ,6.280], present the answer in the form of a list,

Flat_Floor: You can only choose one from ["True", "False"], present the answer in the form of a list,

Sink: You need to give a parameter value in the range [0, 1], present the answer in the form of a list, ...],

Add_Valleys: You can only choose one from ["True", "False"], present the answer in the form of a list,

Scale_Valleys: You need to give a parameter value in the range [0.000, 5.000], present the answer in the form of a list...].

Examples: Inputs: Construct a summer terrain of flat and vast grasslands. Outputs: [Rotate_Slope: [0.000],

Flat_Floor: ["True"], Sink: [0.500], Add_Valleys: ["True"], Scale_Valleys: [1.000] ...]

Code:

```
"""python
import Terrain_mesh_generate
def Terrain_mesh_generate(Rotate_Slope, Flat_Floor, Sink, Add_Valleys, Scale_Valleys, ...):
    bpy.context.object.modifiers["Terrain Mix"]["Input_25"] = Rotate_Slope
    bpy.context.object.modifiers["Terrain Mix"]["Input_26"] = Flat_Floor
    bpy.context.object.modifiers["Terrain Mix"]["Input_27"] = Sink
    bpy.context.object.modifiers["Terrain Mix"]["Input_41"] = Add_Valleys
    bpy.context.object.modifiers["Terrain Mix"]["Input_43"] = Scale_Valley
    ...
    return
"""
```

Example:

Question: The summer highland terrain unveils a captivating expanse, with undulating hills and valleys covered in a verdant carpet of lush green grasses, painting a breathtaking tableau of natural beauty.

Solution: Based on the provided document and your description, here are the parameter values you can use for generating the summer highland terrain:

``plaintext

[Rotate_Slope: [3.142], Flat_Floor: ["False"], Sink: [0.750], Add_Valleys: ["True"], Scale_Valleys: [2.500]...]

``

This configuration will help create undulating hills and valleys, allowing the terrain to have a natural appearance while maintaining its captivating beauty.

Code:

```
"""python
    Terrain_mesh_generate(Rotate_Slope=3.142, Flat_Floor=False, Sink=0.750, Add_Valleys=True,
Scale_Valleys=2.500, ...)
"""
```

Figure 12: Prompt example for terrain PCG mesh generation: a template, code snippet, and specific example.

Template:

Role: You as the Blender API operator choose feasible parameter values based on the description. You need to determine the placement parameters for each individual asset

Task: Your task is to break down the user's input into the parameters required for placing objects. The API that you can invoke for this purpose is called "Scatter".

Document:

[{Name: Density,Description: Based on the user input, speculate on the vegetation density coefficient.,Value: 0.02 - 0.50, Default: 0.20},

{Name: Max_Height, Description: Based on the user input, speculate on the maximum height for placing object , represented as a percentage, Value:0 - 1.000, Default:0.800},

{ Name: Min_Height, Description: Based on the user input, speculate on the minimum height for placing object represented as a percentage, Value:0 - 1.000, Default:0.200},

{Name: Max_Slope, Description: Based on the user input, speculate on the maximum degree of inclination for placing objects, Value: 0 - 90, Default= 90},

{Name: Min_Slope, Description: Based on the user input, speculate on the minimum degree of inclination for placing objects, Value: 0 - 90, Default= 0},...]

Format: Out put the modified api in the follow form:

[Density: You need to give a parameter value in the range [0.02,0.50], present the answer in the form of a list,

Max_Height: You need to give a parameter value in the range [0, 1.000], present the answer in the form of a list,

Min_Height: You need to give a parameter value in the range [0, 1.000], present the answer in the form of a list,

Max_Slope : You need to give a parameter value in the range [0, 90], present the answer in the form of a list,

Min_Slope : You need to give a parameter value in the range [0, 90], present the answer in the form of a list,

...

Examples: Inputs: The tree stands bare without leaves, its branches spreading out in all directions. Outputs:

[Density: [0.2], Max_Height: [1.0], Min_Height: [0.6], Max_Slope : [0], Min_Slope : [60]]

Code:

```
"""python
import Scatter
def Scatter(object, Density, Max_Height, Min_Height, Max_Slope, Min_Slope, ...):
    node_group = bpy.data.node_groups["Density_Scatter_3"]
    node_group.nodes["Density_Scatter_slope"].inputs[5].default_value = math.radians(Min_Slope)
    node_group.nodes["Density_Scatter_slope"].inputs[7].default_value = math.radians(Max_Slope)
    node_group.nodes["Density_Scatter_elev"].inputs[5].default_value = Min_Height
    node_group.nodes["Density_Scatter_elev"].inputs[7].default_value = Max_Height
    ...
    return
"""
```

Example:

Question: The tree stands adorned with lush foliage, its branches reaching upwards in unison.

Solution: Based on the provided document and your description, here are the parameter values you can use for placing objects, assuming the tree as the object:

```plaintext

[Density: [0.20], Max\_Height: [1.000], Min\_Height: [0.200], Max\_Slope: [90], Min\_Slope: [0], ...]

```

This configuration will help place the tree objects with a moderate density, extending from the minimum height of 20% to the maximum height of 100%, and with no restriction on the slope, allowing the branches to spread out in all directions as described.**Code:**

```
"""python
Scatter(obj, Density=0.20, Max_Height=1.000, Min_Height=0.200, Max_Slope=90, Min_Slope=0, ...)
"""
```

Figure 13: Prompt example for object placement: a template, code snippet, and specific example.